COMP 512 Project 2 – Paxos Performance Analysis

Group 19

1 Test Setup and Methodology

System under test. Every move is ordered by its own Paxos instance (slot). A proposer picks the lowest undecided slot and a ballot (*round*, `host:port`), broadcasts PROPOSE, waits for a majority of promises, broadcasts ACCEPT? and waits for a majority of acknowledgements, then broadcasts CONFIRM. An attempt is abandoned when more than $n - \lceil (n+1)/2 \rceil$ acceptors refuse, when another CONFIRM fills the slot, or after a 1 s timeout; the proposer then sleeps a uniformly random 0–50 ms and retries with a round above the highest refused one. `broadcastTOMsg` blocks until the move is decided.

Experiments. We used the provided `TreasureIslandAppAuto` through a parameterised copy of `run_tiapp_auto.sh`, all players on one host (Apple M3, 8 cores, 16 GB, macOS, OpenJDK 21, GCL over loopback, default `FINE` logging). *Sweep:* $n \in \{2,3,5,7,9\} \times$ interval $\in \{0,1,2,5,10,20,50,100,200\}$ ms, 100 moves per player, 2 seeds (90 runs). *Fairness:* $n \in \{5,9\} \times$ interval $\in \{0,10\}$ ms, 600 moves per player, 5 seeds (20 runs). After every run a script compares the move sequences and final islands of all players. All 90 sweep runs and 15 fairness runs were identical. In the other 5 ($n = 9$) one player exited after delivering a *prefix* of the common sequence, as the test app shuts down a fixed 5 s after its own last move.

Metrics. Paxos logs `System.nanoTime()` when `broadcastTOMsg` is called and when it returns, the number of attempts the move needed, and the time each slot is decided. *Latency* is call-to-return time (until a majority accepted the move); each player's first 5 moves (connection set-up) are excluded. *Throughput* is decided slots per second at player 1 between the 10th and 90th percentile of decision times. A *streak* is a run of consecutive slots decided for the same player. For fairness, a player's *acceptance rate* is the number of its moves decided per second while all players are still active (from the end of warm-up until the first player finishes).

2 Impact of the Number of Players and the Interval

Time until a move is accepted – data (Fig. 1, 2b). At 200 ms the mean latency is 6–8 ms for $n = 2$ and 8–25 ms for $n \geq 3$ (median ≈4–7 ms, p95 33–73 ms), with 1.0–1.7 attempts per move. As the interval shrinks to 5–20 ms, latency grows, and faster the more players there are: the mean peaks at 2–6 ms for $n = 2$, 13–14 ms for $n = 5$ and 31–36 ms for $n = 9$ (p95 up to 205 ms), and attempts per move rise to 1.2 ($n = 3$), 1.4 ($n = 5$) and 2.0 ($n = 9$). Below 5 ms latency drops again: the median is ≈2 ms for every n, 98–99% of moves need a single attempt, and for $n = 9$ the mean is 11–20 ms while the p95 is only 9–16 ms (max. 3 s).

Discussion. An uncontested instance costs two round trips to a majority plus synchronous logging of every GCL message, ≈2 ms when processes are busy and 4–7 ms when they idle between moves (threads must first be woken up). Everything above that comes from collisions: only one proposer wins a slot, and every loser pays a random backoff of up to 50 ms, which is where the p95 sits. Collisions occur even at 200 ms because all players start together and wait the same interval, so they keep generating moves at nearly the same instants. At 5–20 ms moves of different players overlap constantly, and more players mean

more proposers per slot, hence more lost attempts and more backoffs. Below 5 ms the system switches to *streaming* (Fig. 3a): a winner proposes its next move immediately, while the losers are still in their backoff, so it wins the next slot uncontested, and the next. On average 19–90 consecutive slots go to the same player (vs. 1.0–1.7 at $\geq$10 ms). Most moves are thus fast, but a player that keeps colliding with a streaming winner can lose dozens of rounds (up to 72 attempts), which explains why the mean exceeds the p95.

Rate at which moves are accepted – data (Fig. 2a). At 200 ms the system accepts 9.6, 14.8, 24.5, 34.0 and 43.3 moves/s for $n = 2,3,5,7,9$, i.e. $\approx$4.8 moves/s per player. At 50 ms this becomes 35, 52, 91, 120 and 139 moves/s. At 5–10 ms all $n \geq 5$ reach the same $\approx$200–230 moves/s, and at 0–2 ms throughput rises again to $\approx$270–475 moves/s for $n \geq 3$ and 380–790 moves/s for $n = 2$.

Discussion. For large intervals Paxos is not the bottleneck: throughput equals the rate at which moves are generated, $n \cdot 1000/(\text{interval} + \delta)$, where $\delta \approx 5$ ms is the per-move overhead of `Thread.sleep` overshoot and logging (43.9 predicted vs. 43.3 measured for $n = 9$ at 200 ms). It therefore grows linearly with n and inversely with the interval. As the interval shrinks, collisions cap the rate below the offered load, and earlier for more players: $n = 9$ is already $\approx$15% below its generation rate at 50 ms, and for $n \geq 5$ halving the interval from 10 to 5 ms no longer increases throughput. The rise at 0–2 ms is the streaming effect above: back-to-back uncontested instances by one player are cheaper than a stream of collisions.

3 Fairness Between Processes

At large intervals every player is accepted at its generation rate; for short ones Table 1 and Fig. 4 show two effects.

Interval 0: strongly unfair in the short term, but not consistently for one process. Streaks are so long (214 slots for $n = 5$, 66 for $n = 9$) that within the window where everybody competes one or two players get most slots. Jain's fairness index is only 0.25–0.45 for $n = 5$ and 0.49–0.82 for $n = 9$. However, the favoured player changes from seed to seed. The aspect responsible is the asymmetric retry policy: losers sleep a random 0–50 ms, but a winner never waits before its next proposal. A streak typically ends only when a returning loser happens to collide with the winner using a higher round. With more players losers return more often, so streaks are shorter for $n = 9$.

Interval 10 ms: almost fair, but with a consistent bias towards higher player numbers. Jain's index is ≥ 0.98, but the acceptance rate increases monotonically with the player number in nearly every run. Player n is 27–28% faster than player 1 (46.9 vs. 36.8 moves/s for $n = 5$, 30.1 vs. 23.6 for $n = 9$), needs fewer attempts and has lower latency, and players 4–5 (resp. 7–9) were the fastest in all seeds. The responsible aspect is the ballot tie-break. After a refusal a proposer jumps to the highest refused round plus one, so colliding proposers very often choose the *same* round, and then `Ballot.compareTo` falls back to the `host:port` string. Ports grow with the player number, so the higher player wins every tie and the lower one pays a backoff. Breaking ties with a random number, or rotating the priority with the slot number, would remove this bias. Letting a winner also back off briefly would shorten the streaks at interval 0.

4 Maximum Throughput

At interval 0 the maximum is $\approx$670 moves/s for $n = 2$, and 340–440 moves/s for $n = 3$–9 (Fig. 3b; seed-to-seed variation up to $\pm$20%). Because of streaking, interval 0 behaves like one leader at a time deciding slots back-to-back. Its rate is bounded by one uncontested instance ($\approx$2–3 ms), which gets slower as n grows since an instance costs $\approx 5n$ messages and all JVMs share 8 cores. The rate does *not* need to reach 0 to flatten: for $n \geq 5$ it already plateaus at 5–10 ms, before streaming lifts it.

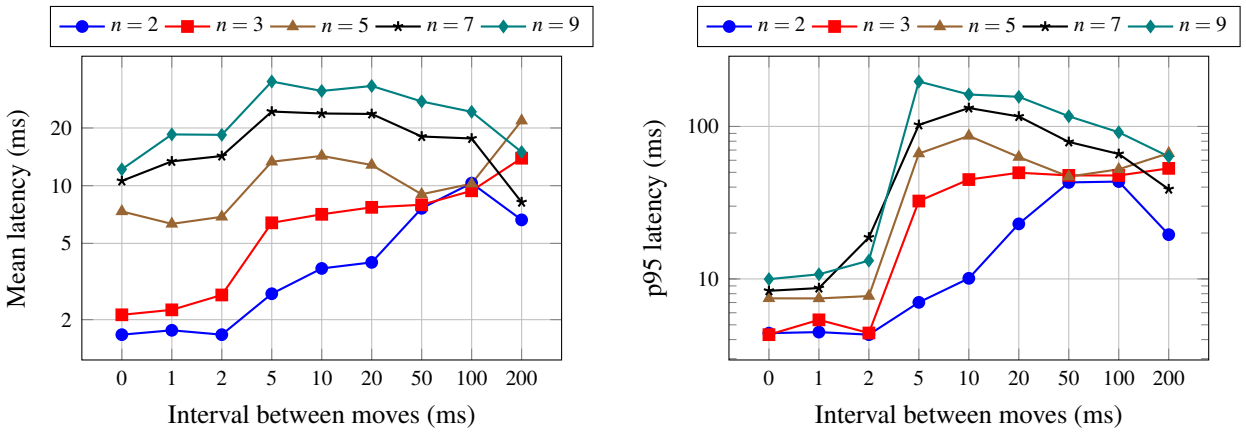

Figure 1: Time from calling `broadcastTOMsg` until the move is accepted by a majority, vs. the interval between moves, for n players. Left: mean; right: 95th percentile (log scale, mean of 2 seeds, 100 moves per player).

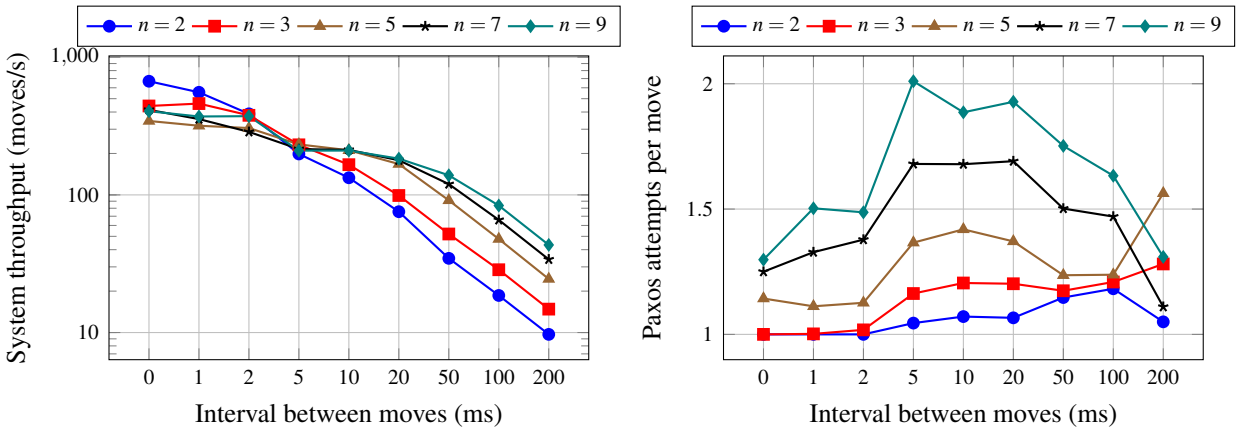

Figure 2: (a) Decided moves per second (log scale). (b) Average number of Paxos attempts (propose rounds) needed per move; 1.0 means no collisions.

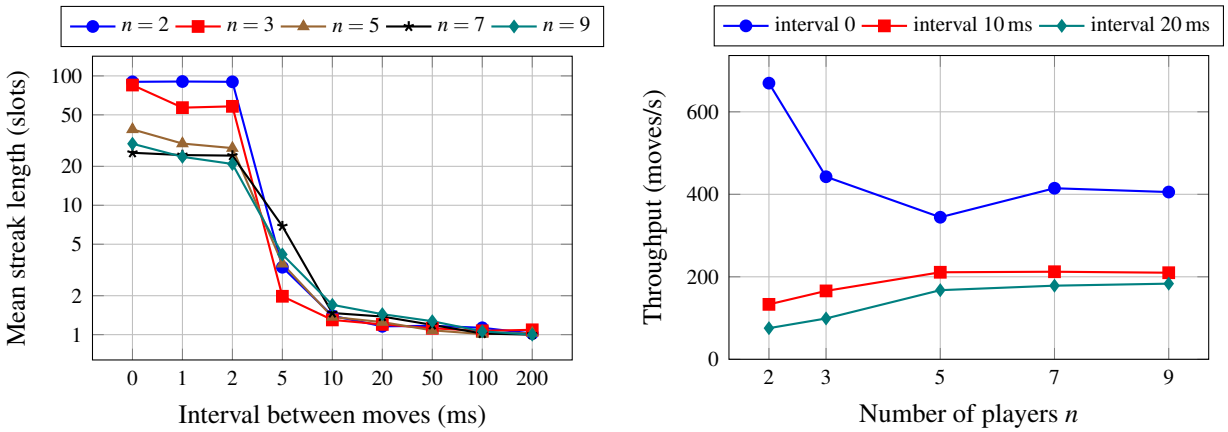

Figure 3: (a) Average number of consecutive slots decided for the same player. (b) Maximum throughput: throughput at interval 0 compared with the contention plateau at 10 and 20 ms.

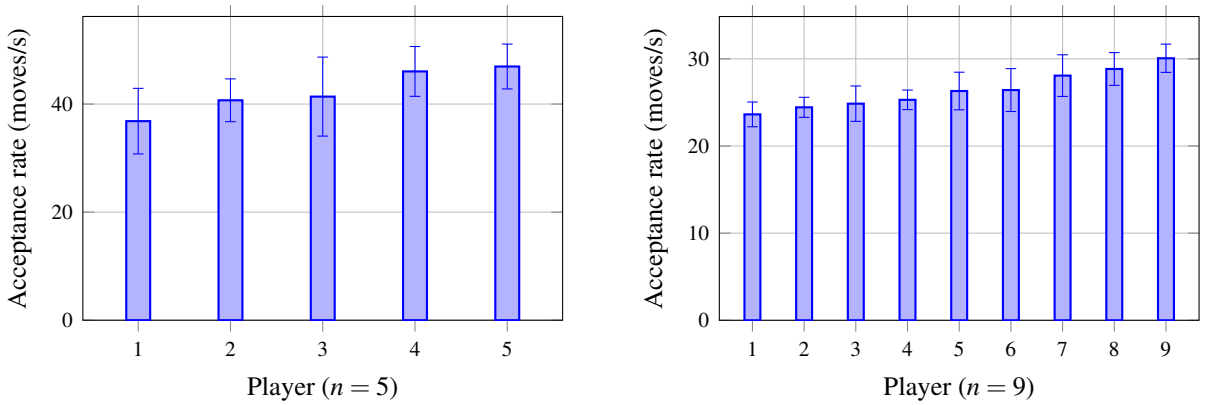

Figure 4: Per-player acceptance rate at interval 10 ms while all players compete (mean $\pm$ std. dev. over 5 seeds, 600 moves per player).

n	interval (ms)	mean streak (slots)	Jain index (min–max)	fastest player (per seed)	rate P1 / Pn (moves/s)	latency P1 / Pn (ms)	attempts P1 / Pn
5	0	213.9	0.25–0.45	2,2,4,4,5	28.3 / 124.8	2.4 / 3.7	1.05 / 1.08
5	10	1.3	0.98–1.00	4,4,5,5,5	36.8 / 46.9	15.8 / 11.9	1.44 / 1.34
9	0	65.6	0.49–0.82	3,3,5,7,8	18.0 / 61.3	10.7 / 10.2	1.31 / 1.26
9	10	1.4	0.98–0.99	7,7,8,9,9	23.6 / 30.1	30.0 / 25.5	1.90 / 1.75

Table 1: Fairness runs (5 seeds each). Jain index $= (\sum r_i)^2/(n\sum r_i^2)$ over per-player acceptance rates (1 = perfectly fair).